Ciberseguridad

Seccion A

Sergio Rolando Oliva del Valle – 26005694

Repo: https://github.com/solivalle/javavulnerablelab-sca

Enlace PR: https://github.com/solivalle/javavulnerablelab-sca/pull/1

Laboratorio: análisis SCA local y en pipeline con Dependency-Check

JavaVulnerableLab

Archivos de la entrega

.github/workflows/sca-dependency-check.yml

entregables/parte B/dependency-check-report.html

entregables/parte C/bom.json

pom.xml

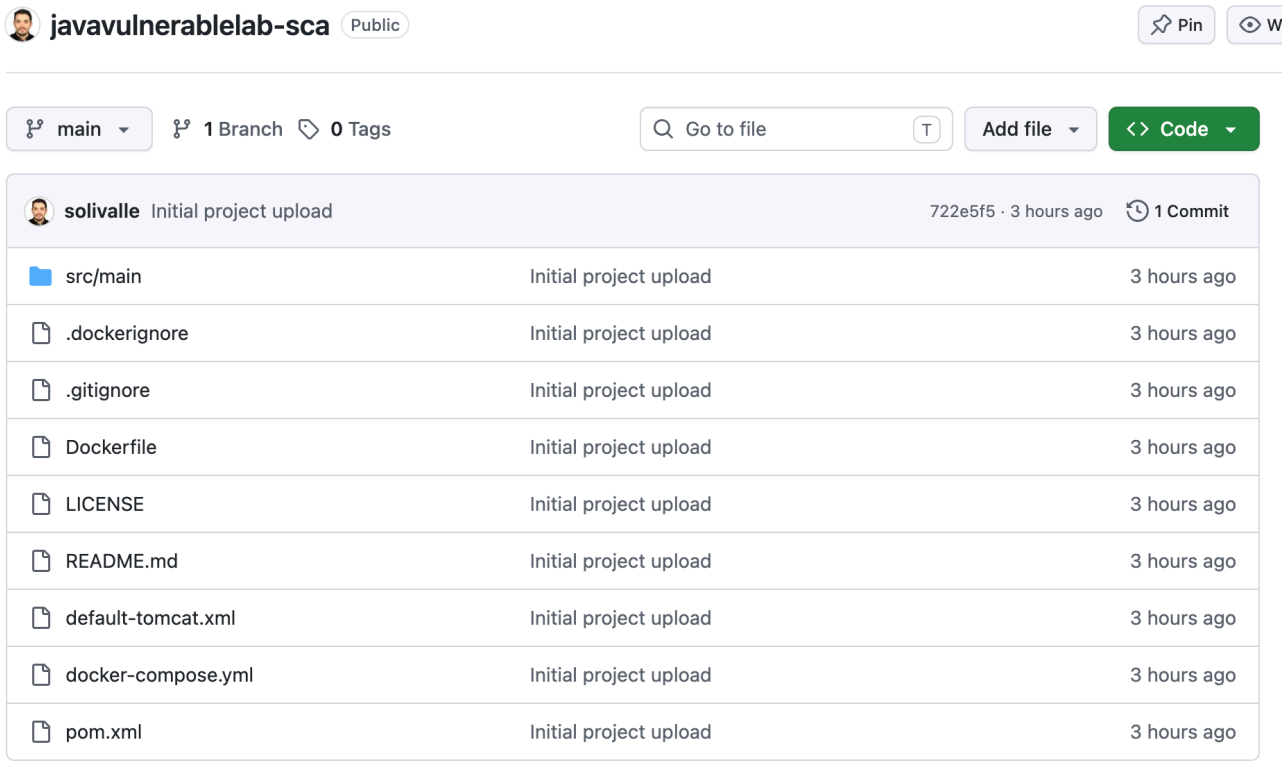
El documento reúne la ejecución local, el inventario CycloneDX, la comparación de herramientas, el pipeline y la remediación. Las preguntas 19 a 23 se presentan como Parte F, que es el encabezado utilizado en las instrucciones; la lista de entregables las menciona como Parte G.

Parte A — Repositorio

Se publicó JavaVulnerableLab en un repositorio público propio. La carpeta target/ contiene salidas de compilación y análisis; las copias de entrega se conservan bajo entregables/. Referencia: entregables/parte A/entregable.md.

Repositorio publicado

Evidencia del repositorio JavaVulnerableLab alojado en GitHub.



Archivo: entregables/parte A/repositorio inicializado.png

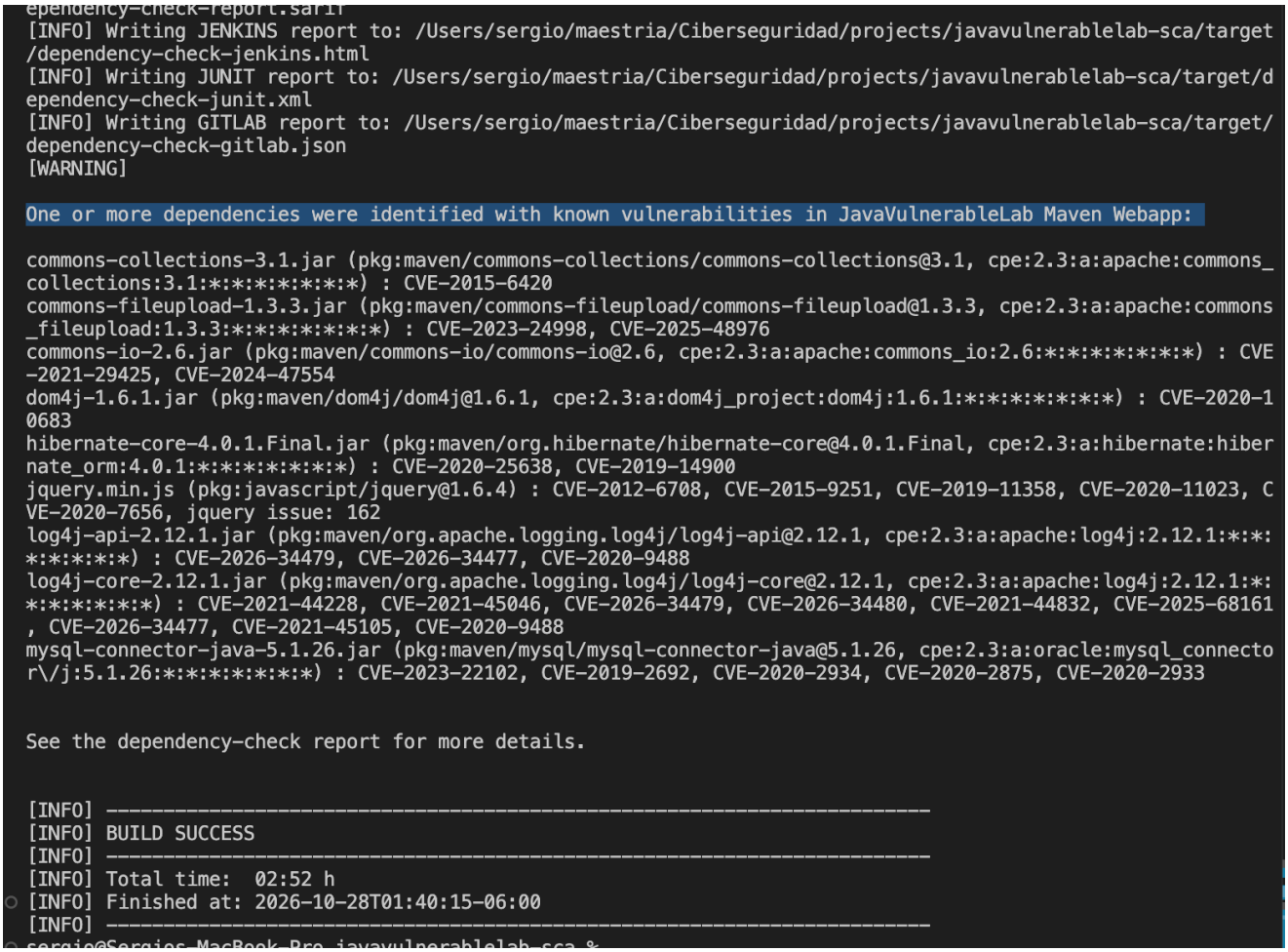
Parte B — Dependency-Check local

Se ejecutó org.owasp:dependency-check-maven:13.0.0:check con formato ALL y la clave del NVD. El reporte inicial registra 19 dependencias analizadas, 9 dependencias vulnerables y 31 hallazgos. OSS Index se deshabilitó en la ejecución local. La clave no se reproduce en esta entrega.

Reportes: HTML y JSON. Procedimiento: entregables/parte B/entregable.md.

Ejecución local en terminal

La salida local muestra los componentes afectados y BUILD SUCCESS. El éxito del comando no significa ausencia de vulnerabilidades.



Archivo: entregables/parte B/dependency-check local.png

Reporte HTML local

Resumen del análisis inicial: 19 dependencias, 9 vulnerables y 31 hallazgos.

Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constitutes acceptance for use in an AS IS condition, and there are NO warranties, implied or otherwise, with regard to the analysis or its use. Any use of the tool and the reporting provided is at the user's risk. In no event shall the copyright holder or OWASP be held liable for any damages whatsoever arising out of or in connection with the use of this tool, the analysis performed, or the resulting report.

How to read the report | **Suppressing false positives** | **Getting Help: github issues**

Project: JavaVulnerableLab Maven Webapp

org.cysecurity:JavaVulnerableLab:0.0.1-SNAPSHOT

Scan Information (show all):

- *dependency-check version*: 13.0.0
- *Report Generated On*: Wed, 28 Oct 2026 01:40:15 -0600
- *Dependencies Scanned*: 19 (19 unique)
- *Vulnerable Dependencies*: 9
- *Vulnerabilities Found*: 31
- *Vulnerabilities Suppressed*: 0
- ...

Summary

Summary of Vulnerable Dependencies (click to show all)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
commons-collections-3.1.jar	cpe:2.3:a:apache:commons_collections:3.1:*:*:*:*:*:*:*	pkg:maven/commons-collections/commons-collections@3.1	CRITICAL	1	Highest	62
commons-fileupload-1.3.3.jar	cpe:2.3:a:apache:commons_fileupload:1.3.3:*:*:*:*:*:*:*	pkg:maven/commons-fileupload/commons-fileupload@1.3.3	HIGH	2	Highest	116
commons-io-2.6.jar	cpe:2.3:a:apache:commons_io:2.6:*:*:*:*:*:*:*	pkg:maven/commons-io/commons-io@2.6	MEDIUM	2	Highest	117
dom4j-1.6.1.jar	cpe:2.3:a:dom4j_project:dom4j:1.6.1:*:*:*:*:*:*:*	pkg:maven/dom4j/dom4j@1.6.1	CRITICAL	1	Highest	120
hibernate-core-4.0.1.Final.jar	cpe:2.3:a:hibernate:hibernate_orm:4.0.1:*:*:*:*:*:*:*	pkg:maven/org.hibernate/hibernate-core@4.0.1.Final	HIGH	2	Low	35
jquery.min.js		pkg:javascript/jquery@1.6.4	MEDIUM*	6		3
log4j-api-2.12.1.jar	cpe:2.3:a:apache:log4j:2.12.1:*:*:*:*:*:*:*	pkg:maven/org.apache.logging.log4j/log4j-api@2.12.1	MEDIUM	3	Highest	42
log4j-core-2.12.1.jar	cpe:2.3:a:apache:log4j:2.12.1:*:*:*:*:*:*:*	pkg:maven/org.apache.logging.log4j/log4j-core@2.12.1	CRITICAL*	9	Highest	46
mysql-connector-java-5.1.26.jar	cpe:2.3:a:oracle:mysql_connector\/j:5.1.26:*:*:*:*:*:*:*	pkg:maven/mysql/mysql-connector-java@5.1.26	HIGH	5	Highest	37

* indicates the dependency has a known exploited vulnerability

Archivo: entregables/parte B/dependency-check html.png

Parte C — SBOM y Dependency-Track

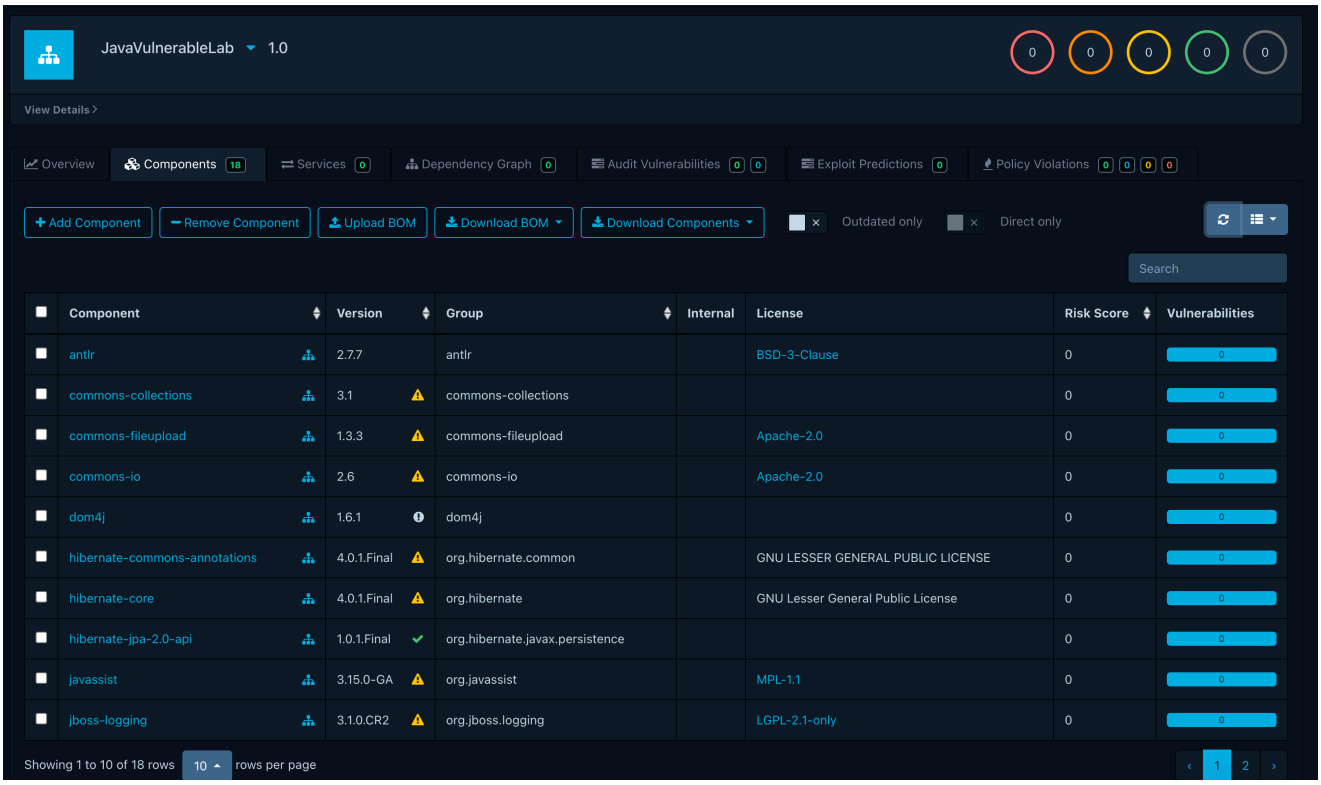
Se generó un SBOM CycloneDX 1.6 en JSON con makeAggregateBom. El inventario contiene 18 componentes, además del componente raíz. Se cargó en el proyecto JavaVulnerableLab, versión 1.0, de Dependency-Track.

Una dependencia transitiva identificada es JavaVulnerableLab → hibernate-core:4.0.1.Final → dom4j:1.6.1. Dependency-Check asocia dom4j con CVE-2020-10683, CRITICAL, CVSS 3.1 de 9.8. El grafo demuestra la relación, mientras que el CVE procede del reporte local.

Referencias: entregables/parte C/bom.json y entregables/parte C/entregable.md.

Componentes del proyecto

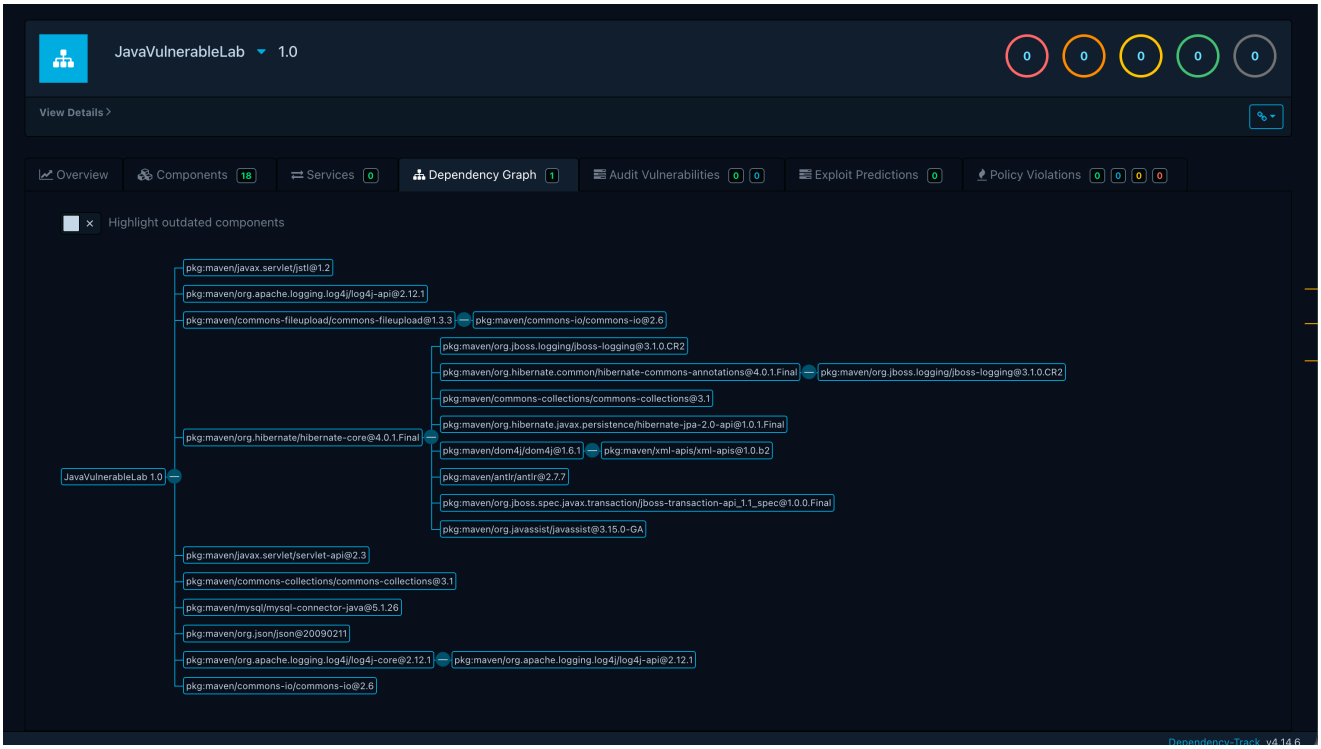
El proyecto registra 18 componentes. Los contadores de vulnerabilidades de esta captura están en cero.



Archivo: entregables/parte C/components.png

Origen de una dependencia transitiva

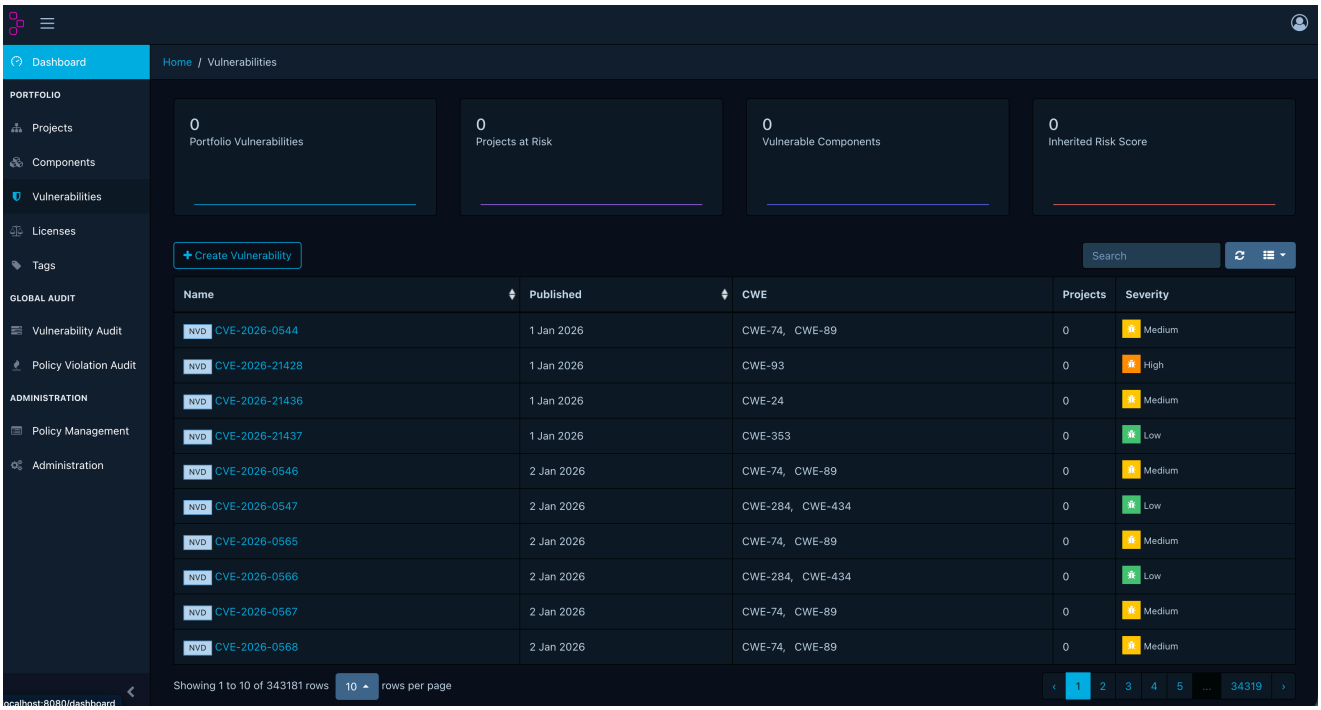
El grafo muestra dom4j:1.6.1 como dependencia de Hibernate.



Archivo: entregables/parte C/dependency graph.png

Vista adicional de Dependency-Track

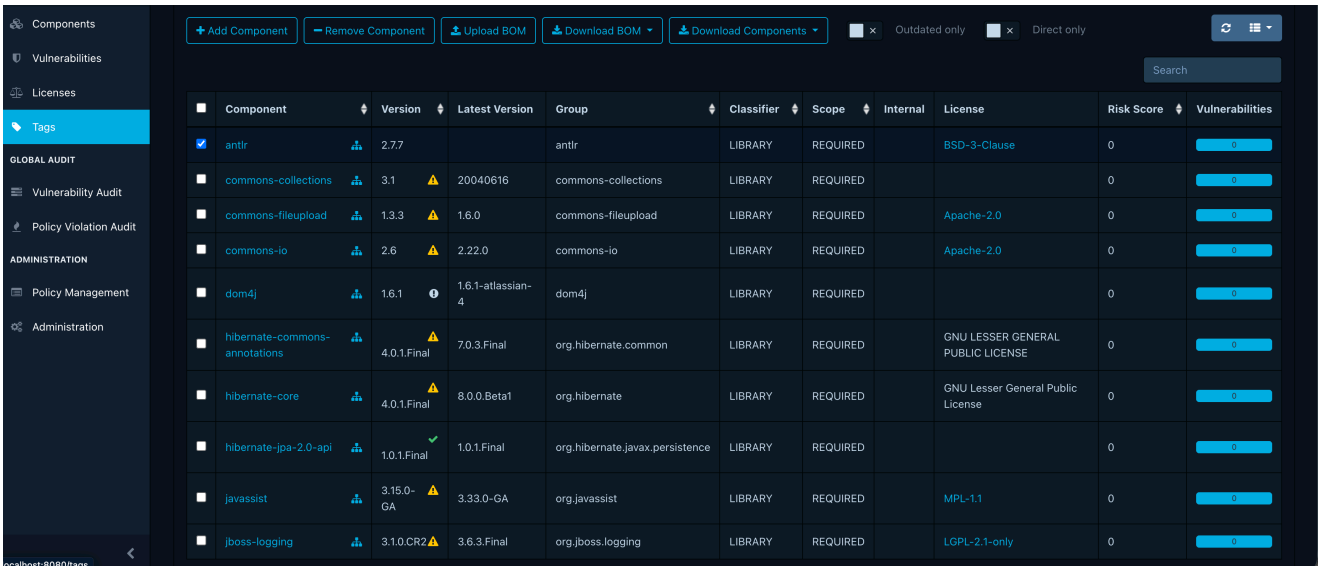
Esta captura corresponde a Vulnerabilities del portafolio global. No corresponde a la pestaña Overview del proyecto JavaVulnerableLab.



Archivo: entregables/parte B/dependency-track overview.png

Inventario adicional en Dependency-Track

Esta captura muestra componentes; no corresponde a la pestaña Audit Vulnerabilities del proyecto. Las dos vistas específicas solicitadas no se acreditan con estas imágenes.



Archivo: entregables/parte B/dependency-track audit.png

Parte D — DefectDojo y comparación

Se creó el producto JavaVulnerableLab y se importó el XML de Dependency-Check con el tipo Dependency Check Scan. La evidencia disponible muestra 31 hallazgos. Los CSV permiten comparar componentes y hallazgos, que son unidades distintas.

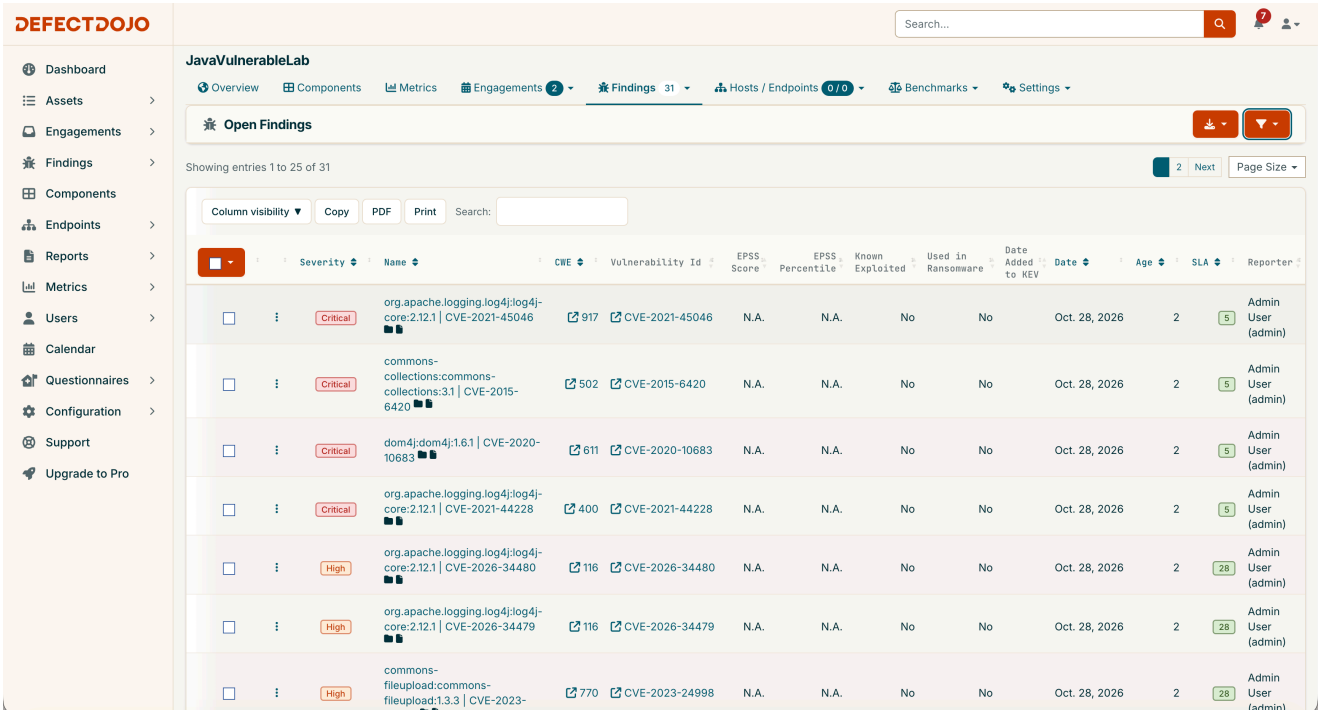
DefectDojo contiene 31 hallazgos distribuidos en 9 componentes: 4 críticos, 7 altos, 16 medios y 4 bajos. Ocho componentes también aparecen en el inventario de Dependency-Track. jQuery 1.6.4 aporta 6 hallazgos en DefectDojo y no figura en el SBOM Maven de 18 componentes.

El CSV de Dependency-Track lista 18 componentes y su columna metrics.vulnerabilities está vacía. Por ello, la coincidencia del inventario no demuestra coincidencia de vulnerabilidades. Las diferencias pueden deberse a cobertura de archivos, fuentes habilitadas, actualización de datos e identificación de componentes. Para comparar CVE se necesitan los hallazgos de ambas herramientas bajo condiciones equivalentes.

Archivos: CSV de DefectDojo y CSV de Dependency-Track.

Hallazgos en DefectDojo

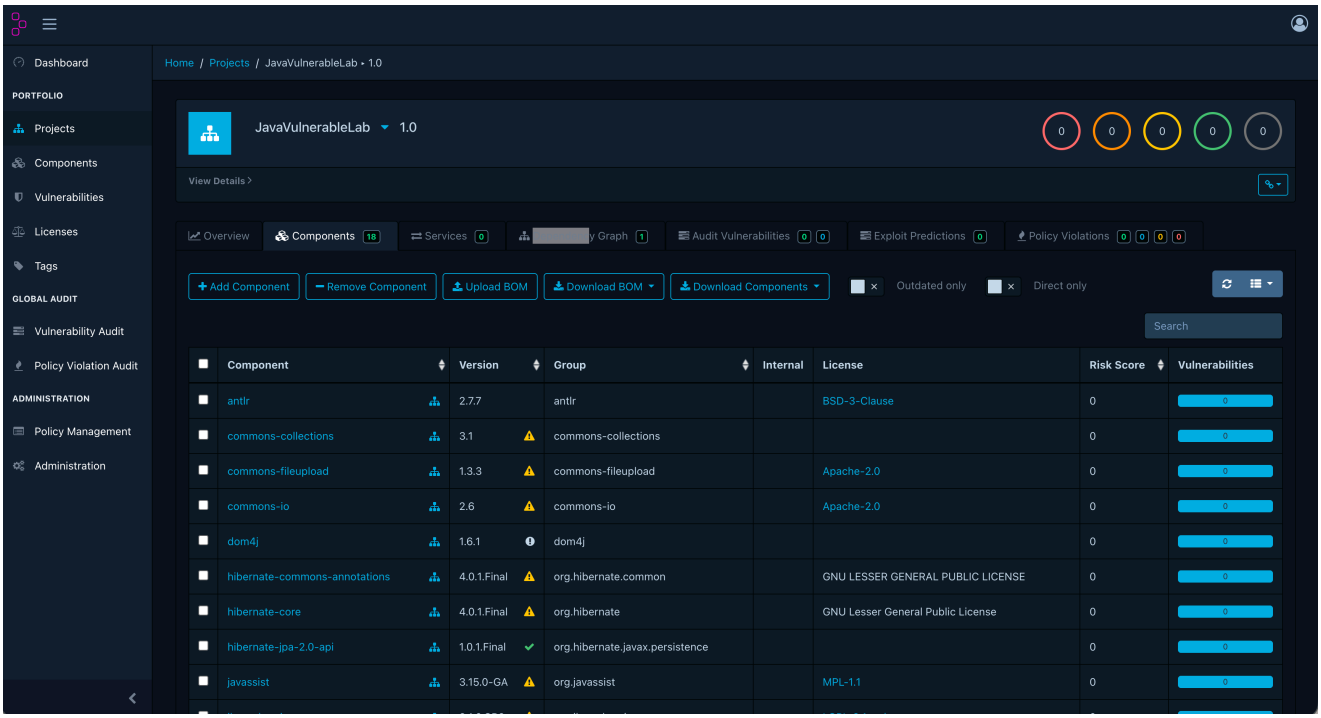
El producto JavaVulnerableLab presenta 31 hallazgos. La captura no muestra el detalle del tipo de escaneo del test importado.



Archivo: entregables/parte D/DefectDojo findings.png

Componentes en Dependency-Track

La vista muestra el inventario y contadores de vulnerabilidades en cero para los componentes visibles.



Archivo: entregables/parte D/Dependency-track findings.png

Parte E — Pipeline de GitHub Actions

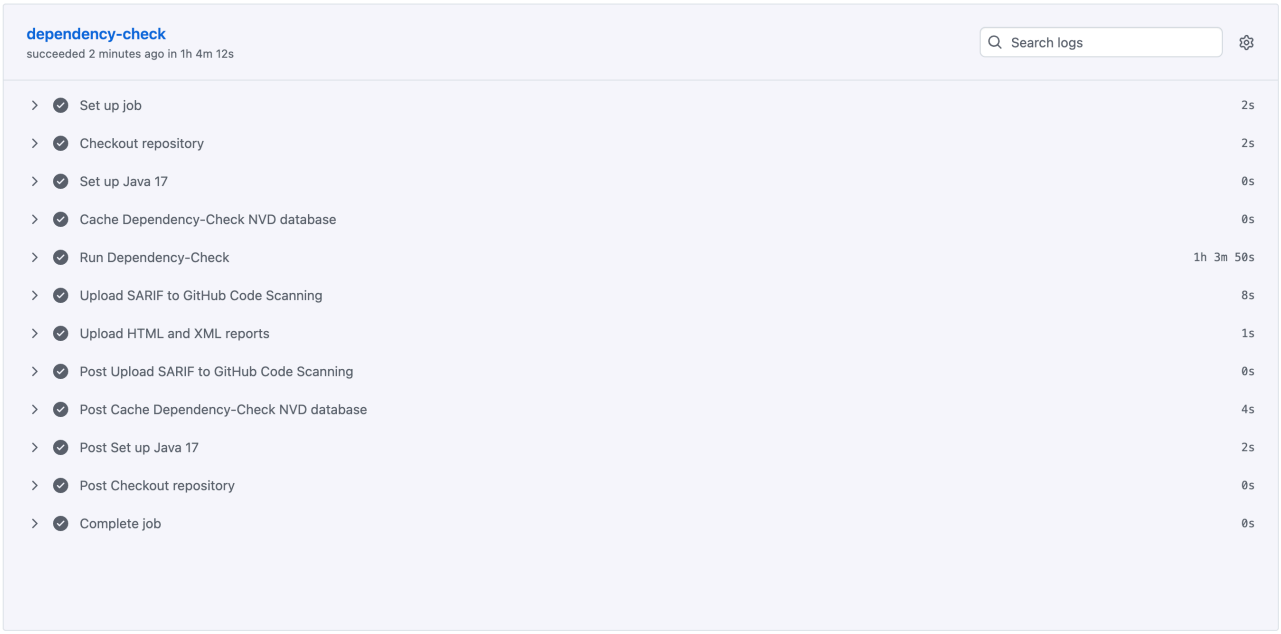
El workflow se activa en push a main, pull_request hacia main y workflow_dispatch. Define contents: read y security-events: write. Todas sus líneas incluyen comentarios explicativos.

1. **Descarga del código:** actions/checkout@v4 obtiene el commit analizado.
2. **Preparación de Java:** actions/setup-java@v5 configura Temurin Java 17 y caché Maven.
3. **Secreto NVD:** NVD_API_KEY se obtiene de secrets.NVD_API_KEY y se entrega al plugin mediante nvdApiKeyEnvironmentVariable; su valor no aparece en el YAML.
4. **Caché NVD:** actions/cache@v4 conserva ~/.dependency-check-data. Maven usa la misma ruta; restore-keys permite reutilizar una caché compatible y actualizarla.
5. **Análisis:** Dependency-Check 13.0.0 ejecuta check con -Dformats=HTML,XML,SARIF y escribe los reportes en target/.
6. **Code scanning:** upload-sarif@v3 publica target/dependency-check-report.sarif con categoría dependencycheck. Se omite la publicación para PR procedentes de forks.
7. **Artefactos:** upload-artifact@v4 conserva dependency-check-report.* en dependency-check-reports. if: always() permite intentar la carga aunque un paso anterior falle.
8. **Job adicional de SBOM:** cyclonedx-sbom se ejecuta de manera independiente con Java 17 y contents: read. CycloneDX Maven Plugin 2.9.3 genera target/bom.json y lo publica como cyclonedx-sbom; falla si el archivo no existe.

Referencias: Workflow comentado y Explicación detallada.

Workflow ejecutado

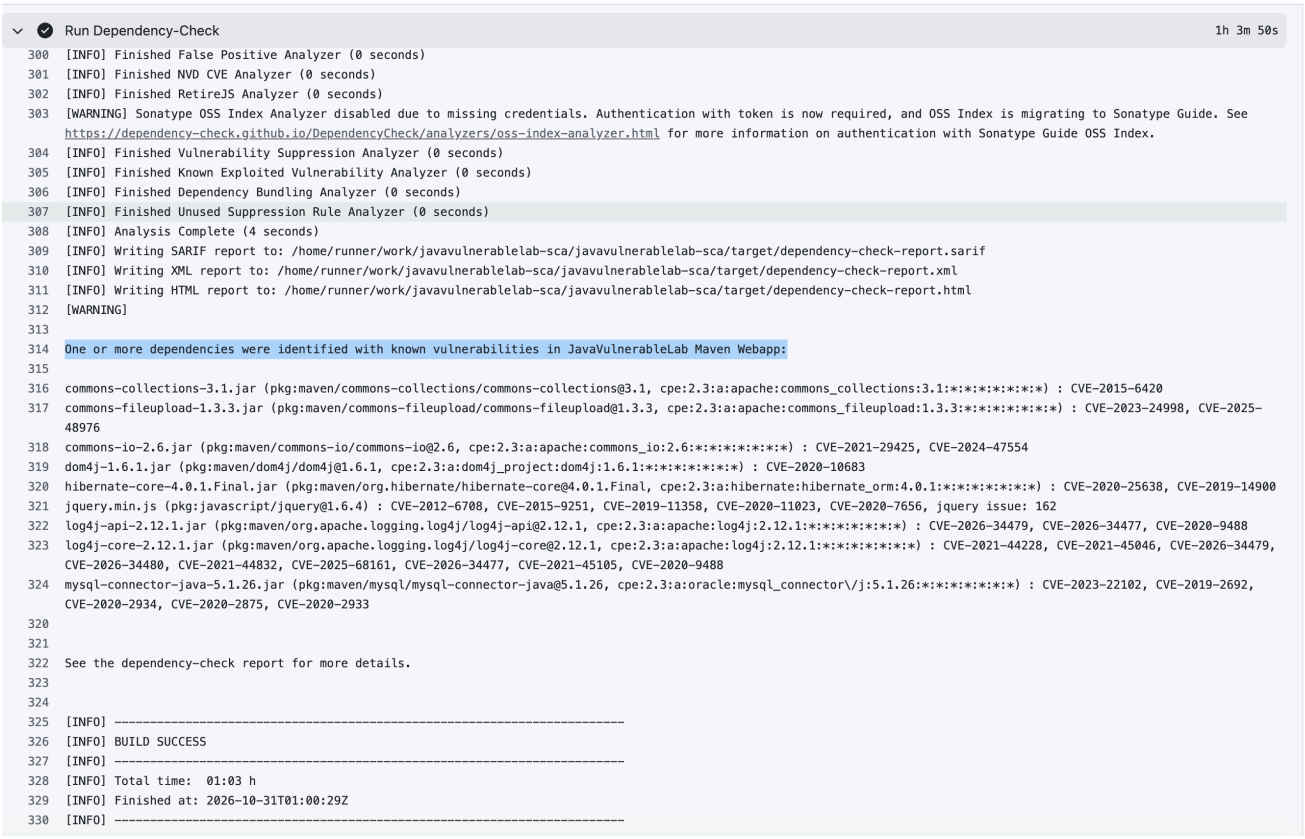
El job dependency-check terminó correctamente en 1 h 4 min 12 s.



Archivo: entregables/parte E/Gitflow Workflow Pipeline.png

Generación de reportes en CI

El log confirma la escritura de HTML, XML y SARIF y BUILD SUCCESS. OSS Index no participó por falta de credenciales.



Run Dependency-Check 1h 3m 50s

```
[INFO] Finished False Positive Analyzer (0 seconds)
[INFO] Finished NVD CVE Analyzer (0 seconds)
[INFO] Finished RetireJS Analyzer (0 seconds)
[WARNING] Sonatype OSS Index Analyzer disabled due to missing credentials. Authentication with token is now required, and OSS Index is migrating to Sonatype Guide. See https://dependency-check.github.io/DependencyCheck/analyzers/oss-index-analyzer.html for more information on authentication with Sonatype Guide OSS Index.
[INFO] Finished Vulnerability Suppression Analyzer (0 seconds)
[INFO] Finished Known Exploited Vulnerability Analyzer (0 seconds)
[INFO] Finished Dependency Bundling Analyzer (0 seconds)
[INFO] Finished Unused Suppression Rule Analyzer (0 seconds)
[INFO] Analysis Complete (4 seconds)
[INFO] Writing SARIF report to: /home/runner/work/javavulnerablelab-sca/javavulnerablelab-sca/target/dependency-check-report.sarif
[INFO] Writing XML report to: /home/runner/work/javavulnerablelab-sca/javavulnerablelab-sca/target/dependency-check-report.xml
[INFO] Writing HTML report to: /home/runner/work/javavulnerablelab-sca/javavulnerablelab-sca/target/dependency-check-report.html
[WARNING]

One or more dependencies were identified with known vulnerabilities in JavaVulnerableLab Maven Webapp:

commons-collections-3.1.jar (pkg:maven/commons-collections/commons-collections@3.1, cpe:2.3:a:apache:commons_collections:3.1:*:*:*:*:*:*:*) : CVE-2015-6420
commons-fileupload-1.3.3.jar (pkg:maven/commons-fileupload/commons-fileupload@1.3.3, cpe:2.3:a:apache:commons_fileupload:1.3.3:*:*:*:*:*:*:*) : CVE-2023-24998, CVE-2025-48976
commons-io-2.6.jar (pkg:maven/commons-io/commons-io@2.6, cpe:2.3:a:apache:commons_io:2.6:*:*:*:*:*:*:*) : CVE-2021-29425, CVE-2024-47554
dom4j-1.6.1.jar (pkg:maven/dom4j/dom4j@1.6.1, cpe:2.3:a:dom4j_project:dom4j:1.6.1:*:*:*:*:*:*:*) : CVE-2020-10683
hibernate-core-4.0.1.Final.jar (pkg:maven/org.hibernate/hibernate-core@4.0.1.Final, cpe:2.3:a:hibernate:hibernate_orm:4.0.1:*:*:*:*:*:*:*) : CVE-2020-25638, CVE-2019-14900
jquery.min.js (pkg:javascript/jquery@1.6.4) : CVE-2012-6708, CVE-2015-9251, CVE-2019-11358, CVE-2020-11023, CVE-2020-7656, jquery issue: 162
log4j-api-2.12.1.jar (pkg:maven/org.apache.logging.log4j/log4j-api@2.12.1, cpe:2.3:a:apache:log4j:2.12.1:*:*:*:*:*:*:*) : CVE-2026-34479, CVE-2026-34477, CVE-2020-9488
log4j-core-2.12.1.jar (pkg:maven/org.apache.logging.log4j/log4j-core@2.12.1, cpe:2.3:a:apache:log4j:2.12.1:*:*:*:*:*:*:*) : CVE-2021-44228, CVE-2021-45046, CVE-2026-34479, CVE-2026-34480, CVE-2021-44832, CVE-2025-68161, CVE-2026-34477, CVE-2021-45105, CVE-2020-9488
mysql-connector-java-5.1.26.jar (pkg:maven/mysql/mysql-connector-java@5.1.26, cpe:2.3:a:oracle:mysql_connector\/j:5.1.26:*:*:*:*:*:*:*) : CVE-2023-22102, CVE-2019-2692, CVE-2020-2934, CVE-2020-2875, CVE-2020-2933

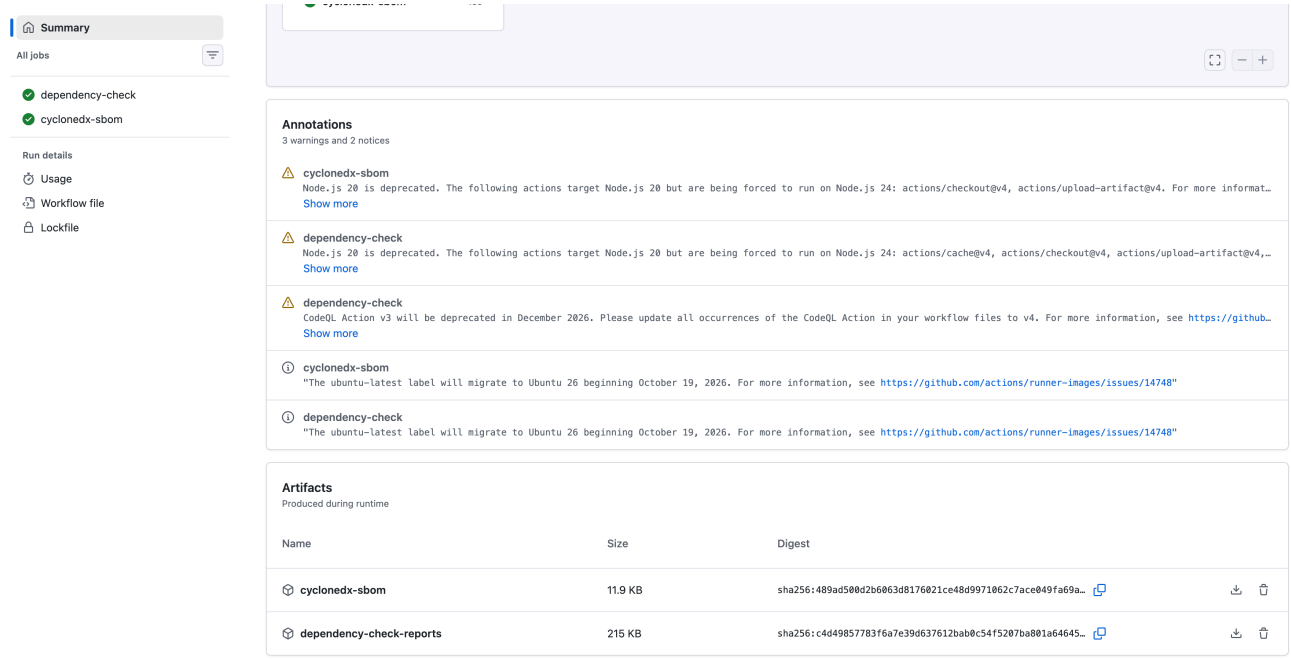

See the dependency-check report for more details.


[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  01:03 h
[INFO] Finished at: 2026-10-31T01:00:29Z
[INFO] ------------------------------------------------------------------------
```

Archivo: entregables/parte E/Evidencia Dependency Check.png

Artefactos descargables

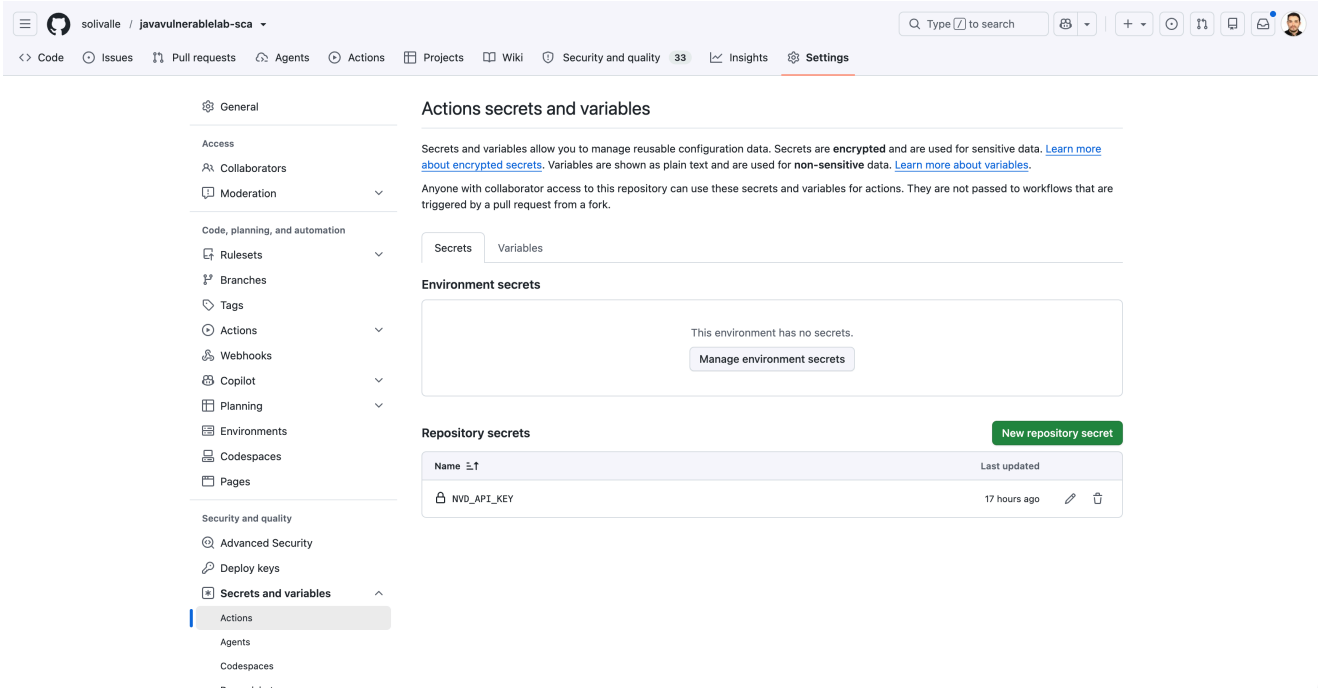
El resumen muestra dependency-check-reports y cyclonedx-sbom, con ambos jobs exitosos.



Archivo: entregables/parte E/Gitflow artifacts.png

Secreto del repositorio

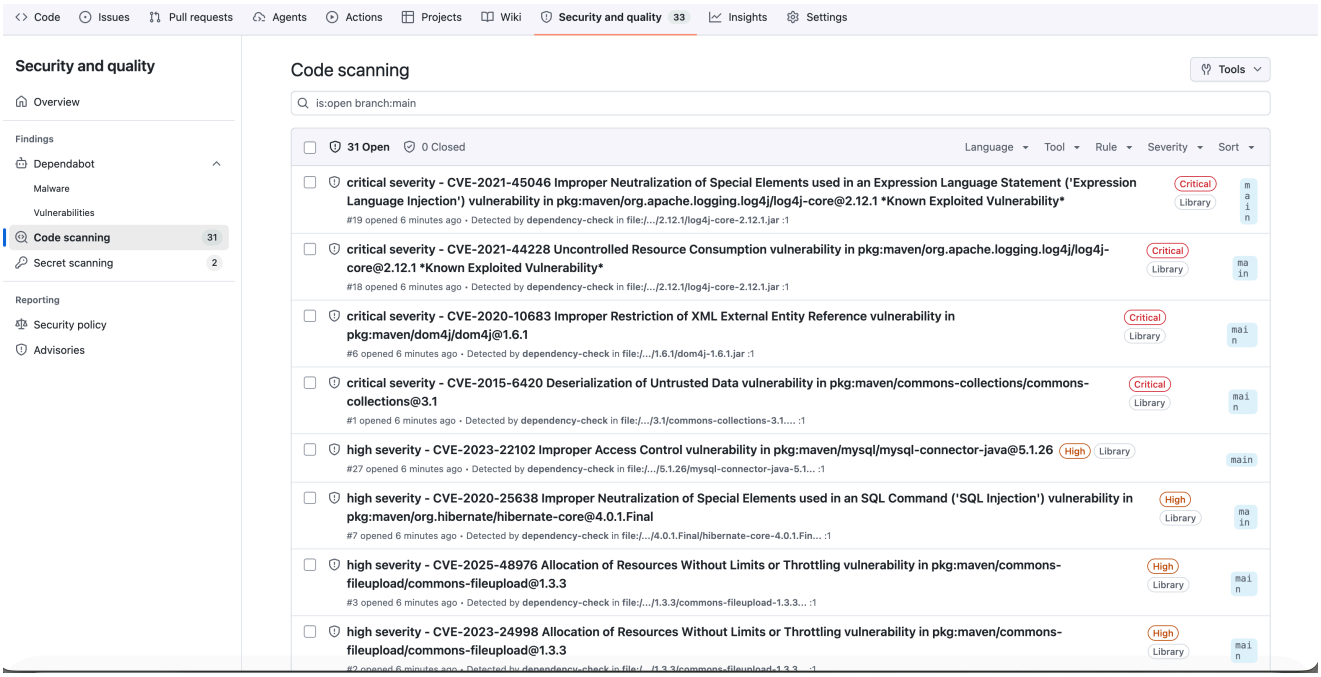
Se confirma el nombre NVD_API_KEY en Repository secrets sin revelar su valor.



Archivo: entregables/parte E/Gitflow NVD_API_KEY.png

Resultados SARIF publicados

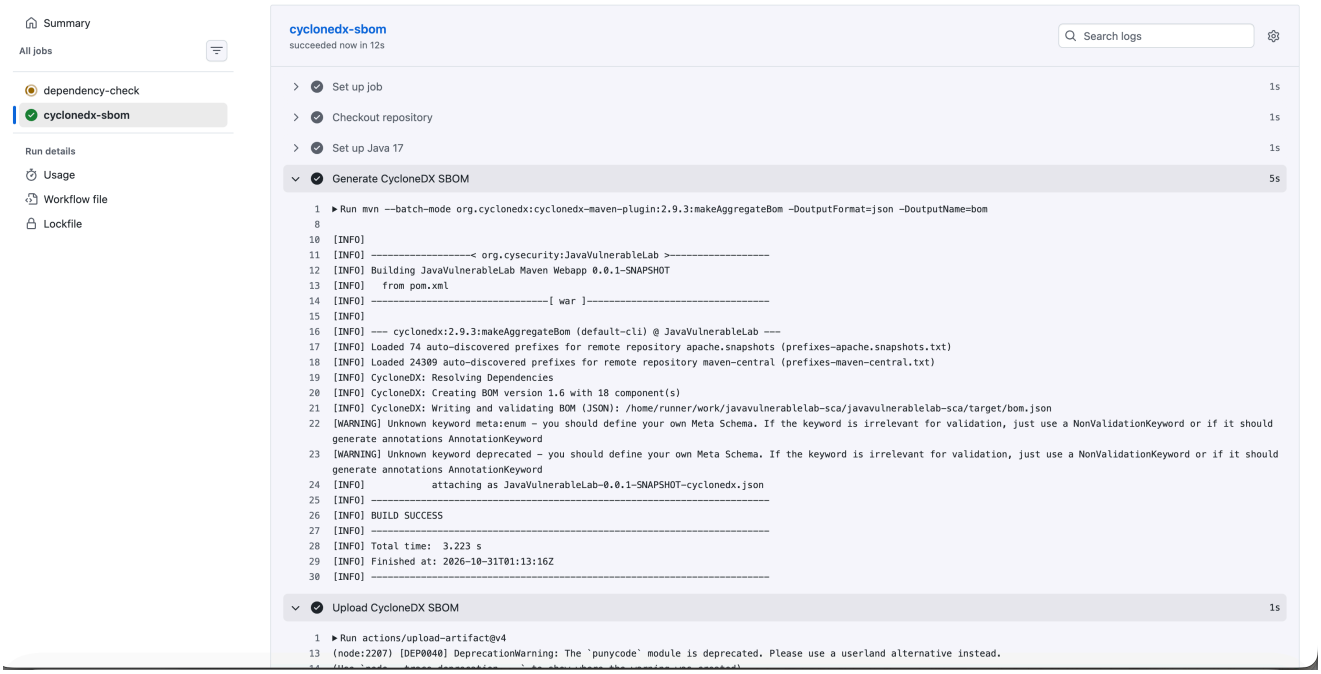
Code scanning muestra 31 alertas abiertas del análisis inicial sobre main.



Archivo: entregables/parte E/Sarif Code Scanning.png

Job adicional CycloneDX

El job generó CycloneDX 1.6 con 18 componentes y terminó correctamente; la carga del artefacto también fue exitosa.



Archivo: entregables/parte E/SBOM CycloneDX.png

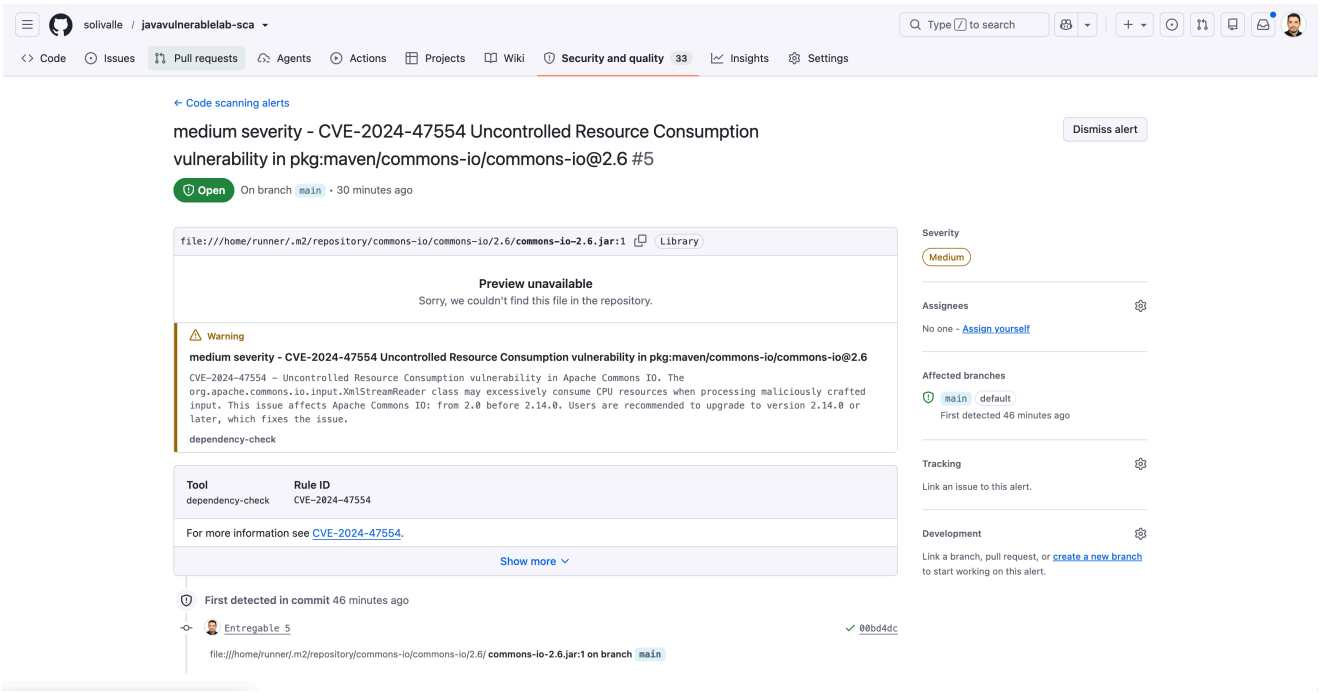
Remediación — Commons IO

Se actualizó commons-io de 2.6 a 2.22.0 en pom.xml. La versión inicial estaba asociada con CVE-2024-47554, consumo excesivo de recursos en XmlStreamReader. La captura posterior muestra la alerta #5 como Fixed en main mediante el pull request #1. Esto acredita la corrección de esa alerta, no la eliminación de todos los hallazgos del proyecto.

Cambio: pom.xml. Pull Request: https://github.com/solivalle/javavulnerablelab-sca/pull/1.

Antes de la remediación

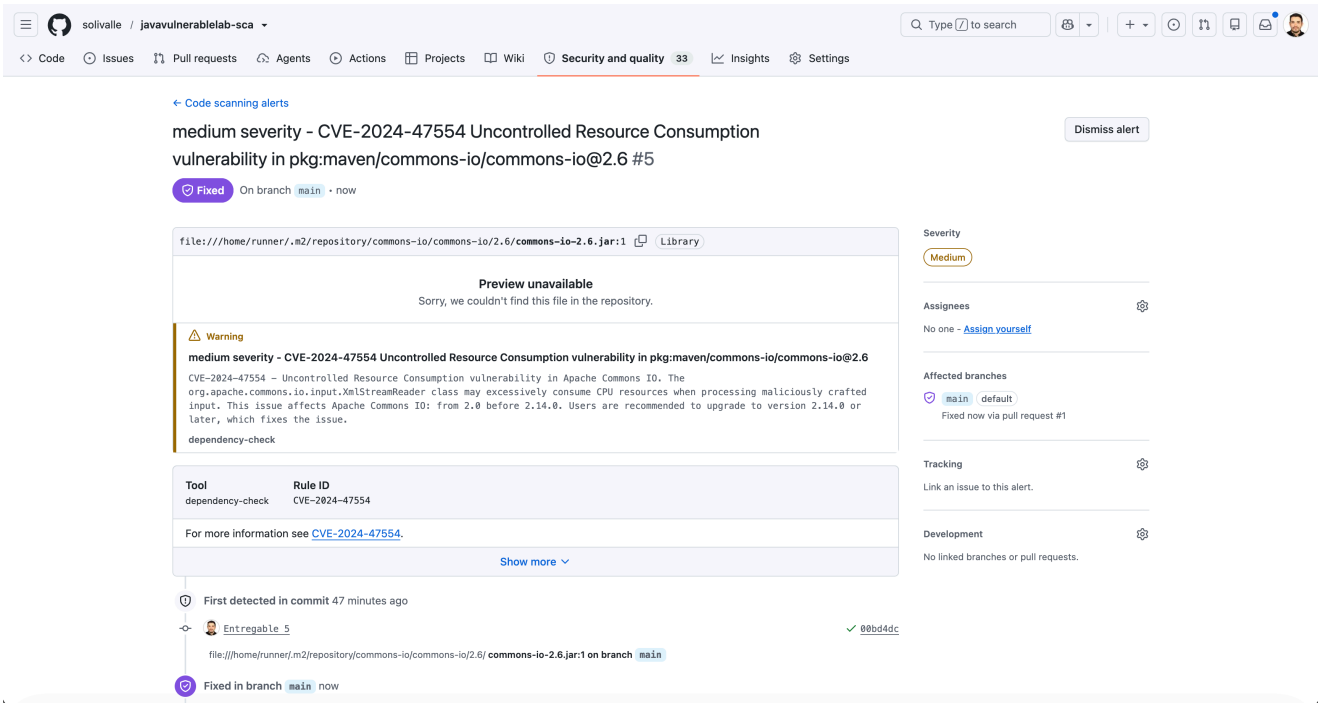
Alerta #5, CVE-2024-47554 en commons-io:2.6, con estado Open en main.



Archivo: entregables/parte F/Vulnerabilidad encontrada en github.png

Después de la remediación

La misma alerta #5 aparece con estado Fixed en main y referencia al pull request #1.



Archivo: entregables/parte F/Vulnerabilidad arreglada en github.png

Parte F — Preguntas de análisis

Las respuestas corresponden a las preguntas 19 a 23 de la tarea y se apoyan en los reportes y evidencias conservados en las partes B, C, D y E.

19. ¿Qué diferencia hay entre SAST y SCA? ¿Por qué una aplicación necesita ambos?

SAST (Static Application Security Testing) analiza estáticamente el código fuente o compilado para identificar posibles defectos de seguridad, por ejemplo una consulta SQL construida con datos de entrada sin tratamiento adecuado. Ayuda a localizar el problema en la implementación de la aplicación. Esta definición corresponde a la descripción de OWASP sobre análisis estático.

SCA (Software Composition Analysis) examina los componentes de terceros que forman parte del software, sus versiones y sus relaciones. Permite identificar dependencias asociadas con vulnerabilidades conocidas y, según la herramienta, revisar también licencias. Dependency-Check se utiliza en este laboratorio para detectar componentes con vulnerabilidades conocidas.

La aplicación necesita ambos porque puede contener fallos en su propio código y también incorporar fallos mediante bibliotecas externas. Corregir una consulta insegura no actualiza una biblioteca vulnerable; actualizar una biblioteca tampoco elimina automáticamente una vulnerabilidad en el código de la aplicación.

En este proyecto, el reporte local identifica `log4j-core:2.12.1` con **CVE-2021-44228**, severidad **CRITICAL** y CVSS 3.1 **10.0**. Este es un resultado de SCA. Un análisis SAST complementario serviría para revisar cómo el código maneja entradas, consultas y operaciones sensibles. Las evidencias entregadas no incluyen una ejecución SAST, por lo que no se atribuyen a esa técnica hallazgos que no fueron comprobados.

20. ¿Qué es una dependencia transitiva y por qué es difícil de controlar? Da un ejemplo de este proyecto.

Una dependencia transitiva es una biblioteca que se incorpora porque otra dependencia la necesita, aunque el proyecto no la declare directamente en su `pom.xml`.

Es difícil de controlar porque el desarrollador suele revisar primero las dependencias directas, mientras que las transitivas pueden formar varias capas. Además, al actualizar una dependencia principal pueden cambiar las versiones resueltas de sus componentes internos. Forzar una versión o excluir una biblioteca requiere comprobar compatibilidad para evitar que la aplicación deje de funcionar.

En JavaVulnerableLab se documentó esta relación:

```
JavaVulnerableLab
└── org.hibernate:hibernate-core:4.0.1.Final
    └── dom4j:dom4j:1.6.1
```

Hibernate está declarado directamente en el pom.xml. `dom4j:1.6.1` llega a través de Hibernate, como muestran las relaciones del SBOM y la captura del Dependency Graph.

El reporte de Dependency-Check asocia esa versión de dom4j con **CVE-2020-10683**, severidad **CRITICAL** y CVSS 3.1 **9.8**, relacionado con el procesamiento de DTD y entidades externas XML. La evidencia del grafo demuestra el origen transitivo; la asociación con el CVE procede del reporte de Dependency-Check, como se explica en la Parte C.

21. ¿Qué ventaja tiene Dependency-Track, con monitoreo continuo del SBOM, frente a ejecutar Dependency-Check solo en el pipeline?

Dependency-Check en el pipeline proporciona un análisis cuando se ejecuta el workflow. En este repositorio ocurre con un push a `main`, un pull request hacia `main` o una ejecución manual. Si después se publica una vulnerabilidad y no se vuelve a ejecutar el pipeline, los reportes anteriores no se actualizan por sí solos.

La ventaja de Dependency-Track es mantener un inventario por proyecto y versión y volver a contrastarlo con información de vulnerabilidades que se actualiza. Así puede detectar un riesgo nuevo en componentes ya inventariados sin necesitar un cambio de código o una compilación nueva. Esto depende de mantener la instancia operativa y sus fuentes de información actualizadas.

En la Parte C se cargó un SBOM con **18 componentes**. Ese inventario permite dar seguimiento a las versiones registradas. Si la composición de la aplicación cambia, es necesario subir un SBOM nuevo para que el seguimiento represente el software realmente utilizado.

El workflow de la Parte E también genera `target/bom.json` y lo guarda como artefacto `cyclonedx-sbom`. Actualmente ese job **no lo envía automáticamente a Dependency-Track**; conservar el artefacto no sustituye la carga del SBOM en la plataforma.

22. ¿Por qué crees que Dependency-Check y Dependency-Track pueden reportar resultados distintos para el mismo proyecto?

Los resultados pueden variar por los siguientes motivos:

• **Alcance del inventario:** Dependency-Check puede identificar archivos que no están incluidos en un SBOM generado a partir de las dependencias Maven.

• **Fuentes y actualización:** las fuentes activas y la fecha de sincronización de sus datos pueden diferir. En la ejecución local se deshabilitó OSS Index y en el pipeline se observa que no participó por falta de credenciales.

• **Identificación de componentes:** diferencias en las evidencias, nombres, versiones e identificadores pueden producir asociaciones distintas con vulnerabilidades, incluidos falsos positivos o negativos.

• **Configuración y procesamiento:** los analizadores habilitados, las supresiones, el alcance de dependencias y el estado del procesamiento pueden cambiar lo que aparece en los resultados. Un

contador en cero requiere revisar el estado de la instancia antes de concluir que no hay vulnerabilidades.

La documentación de Dependency-Check reconoce que algunos analizadores pueden producir falsos positivos y negativos. Por su parte, Dependency-Track contrasta el inventario con múltiples fuentes de información; utilizar ambas herramientas no garantiza resultados idénticos.

En las evidencias de este proyecto se observan dos diferencias concretas:

1. **jQuery:** Dependency-Check identifica `jquery.min.js` versión `1.6.4`. La importación en DefectDojo contiene **6 hallazgos** relacionados con este componente, pero jQuery no figura entre los 18 componentes del SBOM Maven utilizado en Dependency-Track. La cobertura de entrada ya es diferente.

2. **Componentes frente a hallazgos:** el CSV de DefectDojo contiene **31 hallazgos en 9 componentes**, mientras que el CSV de Dependency-Track lista **18 componentes**. La columna `metrics.vulnerabilities` de este último está vacía y las capturas de la Parte C muestran cero vulnerabilidades. Por ello, esos 18 registros no demuestran que Dependency-Track haya identificado 18 componentes vulnerables.

Ocho de los nueve componentes con hallazgos en DefectDojo también aparecen en el inventario de Dependency-Track. Esto confirma coincidencia de componentes, pero no demuestra coincidencia de CVE o severidades. Para comparar detecciones habría que obtener los hallazgos de ambas herramientas con sus fuentes sincronizadas, identificadores y fechas de análisis.

Evidencias: reporte JSON de Dependency-Check, CSV de DefectDojo, CSV de Dependency-Track y documentación de la Parte C.

23. Si una librería vulnerable no tiene versión corregida, ¿qué alternativas tienes para gestionar el riesgo?

Primero revisaría si el hallazgo afecta a la versión utilizada y si las funciones o configuraciones vulnerables están presentes en el despliegue. Que una biblioteca aparezca en el inventario no demuestra por sí solo que una vulnerabilidad sea explotable en todos los contextos; esa evaluación debe quedar documentada.

Si el riesgo aplica y no hay una versión corregida, consideraría estas alternativas:

1. **Eliminar la dependencia o la funcionalidad que la requiere**, cuando no sea necesaria. Si es transitiva, revisar si puede excluirse o si puede sustituirse la biblioteca que la incorpora, comprobando compatibilidad.

2. **Migrar a otra biblioteca mantenida** que cubra la misma necesidad. La sustitución requiere adaptar la integración y verificar el comportamiento de la aplicación.

3. **Aplicar un parche o una mitigación del proveedor**, si existe aunque todavía no se haya publicado una versión nueva. Si se mantiene un parche propio, registrar su origen y asumir el mantenimiento necesario.

4. **Reducir la exposición:** deshabilitar la función vulnerable, restringir entradas y acceso, limitar permisos o aislar el servicio. Los controles deben responder al mecanismo del fallo y verificarse; por ejemplo, ante un riesgo de entidades externas XML, evaluar la desactivación de DTD y entidades externas en el parser utilizado.

5. **Posponer o bloquear el despliegue** cuando el impacto y la exposición no puedan reducirse a un nivel aceptable.

6. **Aceptar temporalmente el riesgo**, si corresponde, con un responsable, una justificación, controles compensatorios, una fecha de revisión y seguimiento de futuras correcciones.

En todos los casos mantendría el componente en el SBOM, registraría la decisión en la gestión de hallazgos y repetiría el análisis cuando cambie la biblioteca, su configuración o la información disponible. Suprimir una alerta solo cambia su presentación; no corrige la vulnerabilidad.

En este laboratorio, `dom4j:1.6.1` sirve como ejemplo de una dependencia transitiva cuya remediación debe considerar también Hibernate. Si hipotéticamente no existiera una actualización compatible, evaluaría la migración del componente que la incorpora y controles sobre el procesamiento XML. Las alternativas para dom4j son hipotéticas. La remediación aplicada a Commons IO se documenta en la sección anterior.